

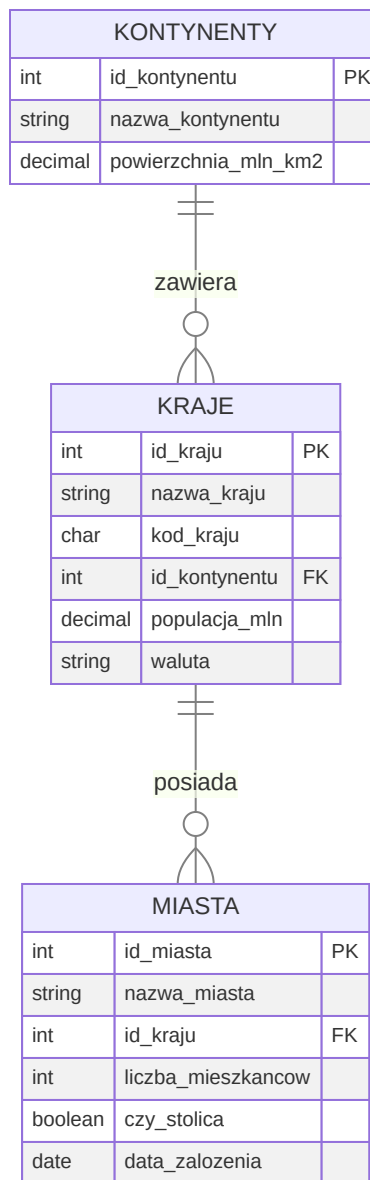
# Laboratorium 6: Funkcje w SQL

## Cel laboratorium

Celem laboratorium jest zapoznanie się z wykorzystaniem wbudowanych funkcji SQL (tekstowych, numerycznych, daty) oraz tworzeniem własnych funkcji (User-Defined Functions - UDF) w systemie PostgreSQL.

## Schemat bazy danych

Laboratorium bazuje na danych o miastach, krajach i kontynentach.



# Podstawy teoretyczne

## 1. Funkcje tekstowe

Pozwalają na manipulację ciągami znaków.

- UPPER(string) , LOWER(string) - zmiana wielkości liter.
- LENGTH(string) - długość ciągu.
- SUBSTRING(string FROM start FOR count) - wycinanie fragmentu.

- CONCAT(s1, s2, ...) lub operator || - łączenie ciągów.
- REPLACE(string, old, new) - zamiana fragmentu tekstu.

## 2. Funkcje numeryczne

Służą do operacji na liczbach.

- ROUND(numer, miejsca) - zaokrąglanie.
- CEIL(numer) , FLOOR(numer) - zaokrąglanie w górę/dół.
- ABS(numer) - wartość bezwzględna.
- RANDOM() - liczba losowa z zakresu [0, 1).

## 3. Funkcje daty i czasu

Umożliwiają pracę z typami czasowymi.

- NOW() - bieżąca data i czas.
- CURRENT\_DATE , CURRENT\_TIME .
- EXTRACT(field FROM source) - pobranie części daty (np. YEAR , MONTH , DAY ).
- AGE(timestamp) - oblicza różnicę czasu od teraz do podanej daty.

## 4. Instrukcja CASE WHEN

Pozwala na stosowanie logiki warunkowej wewnątrz zapytania SQL, co jest odpowiednikiem konstrukcji if-else w językach programowania.

**Przykład:**

```
SELECT nazwa_miasta,
       CASE
         WHEN liczba_mieszkancow > 1000000 THEN 'Metropolia'
         WHEN liczba_mieszkancow > 500000 THEN 'Duże miasto'
         ELSE 'Mniejsze miasto'
       END AS kategoria_miasta
FROM miasta;
```

## 5. Funkcje definiowane przez użytkownika (UDF)

W PostgreSQL funkcje tworzy się za pomocą polecenia `CREATE FUNCTION`. Jeśli funkcja korzysta ze zmiennych lokalnych, należy użyć sekcji `DECLARE` przed blokiem `BEGIN ... END`.

**Przykład 1:** Funkcja przeliczająca populację na miliony z dopiskiem jednostki (prosta funkcja bez zmiennych):

```
CREATE OR REPLACE FUNCTION formatuj_populacje(populacja numeric)
RETURNS text AS $$
BEGIN
    RETURN populacja || ' mln';
END;
$$ LANGUAGE plpgsql;
```

**Przykład 2:** Funkcja obliczająca gęstość zaludnienia z użyciem sekcji `DECLARE` dla zmiennej pomocniczej:

```
CREATE OR REPLACE FUNCTION gestosc_zaludnienia(populacja_mln numeric, p
RETURNS numeric AS $$
DECLARE
    wynik numeric;
BEGIN
    IF powierzchnia = 0 THEN
        RETURN 0;
    END IF;
    wynik := (populacja_mln * 1000000) / powierzchnia;
    RETURN wynik;
END;
$$ LANGUAGE plpgsql;
```

---

**Przykład 3:** Funkcja sprawdzająca, czy miasto jest "duże" (powyżej 1 mln mieszkańców):

```
CREATE OR REPLACE FUNCTION czy_duze_miasto(liczba_mieszkancow int)
RETURNS boolean AS $$
BEGIN
```

```
RETURN liczba_mieszkancow > 1000000;  
END;  
$$ LANGUAGE plpgsql;
```

## Zadania do wykonania

1. Wyświetl nazwy wszystkich miast zapisane wielkimi literami. *(Podpowiedź: użyj UPPER )*
2. Wyświetl nazwy krajów oraz ich kod kraju w nawiasach (np. "Polska (POL)").  
*(Podpowiedź: użyj operatora || lub funkcji CONCAT )*
3. Znajdź miasta, których nazwa ma więcej niż 8 znaków. *(Podpowiedź: użyj funkcji LENGTH w klauzuli WHERE )*
4. Wyświetl nazwy miast i pierwsze trzy litery ich nazwy w osobnej kolumnie.  
*(Podpowiedź: użyj SUBSTRING lub LEFT )*
5. Oblicz, ile lat temu zostało założone każde miasto. *(Podpowiedź: użyj EXTRACT(YEAR FROM AGE(NOW(), data\_zalozenia)) )*
6. Wyświetl nazwy krajów, zamieniając w ich nazwach literę 'a' na '@'.  
*(Podpowiedź: użyj funkcji REPLACE )*
7. Wyświetl nazwy miast oraz kolumnę informującą o liczbie mieszkańców w tysiącach (zaokrągloną do całości). *(Podpowiedź: podziel liczbę mieszkańców przez 1000 i użyj ROUND )*
8. Wyświetl aktualną datę w formacie: "Dzisiaj jest: [data]". *(Podpowiedź: użyj CONCAT i CURRENT\_DATE )*
9. Dla każdego kraju wyświetl informację: "Kraj [nazwa] używa waluty [waluta]".  
*(Podpowiedź: użyj operatora || )*
10. Wyświetl miasta założone w XVIII wieku (lata 1701-1800). *(Podpowiedź: użyj EXTRACT(YEAR FROM ...) lub operatora BETWEEN )*
11. Stwórz funkcję staz\_miasta(id\_miasta\_param int) , która zwraca liczbę lat od założenia miasta do dziś. *(Podpowiedź: wzoruj się na zadaniu 5, użyj SELECT data\_zalozenia INTO ... FROM miasta ... )*
12. Użyj stworzonej funkcji staz\_miasta , aby wyświetlić miasta starsze niż 500 lat. *(Podpowiedź: wywołaj funkcję w klauzuli WHERE )*

13. Wyświetl nazwy kontynentów oraz ich powierzchnię zaokrągloną do jednego miejsca po przecinku. *(Podpowiedź: użyj funkcji ROUND )*
14. Wylosuj dla każdego miasta "liczbę szczęścia" z zakresu od 1 do 100.  
*(Podpowiedź: użyj FLOOR(RANDOM() \* 100) + 1 )*
15. Wyświetl miasta, których nazwa kończy się na literę 'a'. *\*(Podpowiedź: użyj RIGHT(nazwa, 1) lub operatora LIKE )\**
16. Oblicz średnią liczbę mieszkańców dla wszystkich miast, wynik zaokrąglaj do dwóch miejsc po przecinku. *(Podpowiedź: użyj ROUND(AVG(...), 2) )*
17. Stwórz funkcję czy\_stara\_stolica(id\_miasta\_param int) , która zwraca TRUE , jeśli miasto jest stolicą i zostało założone przed 1500 rokiem.  
*(Podpowiedź: użyj instrukcji IF i pobierz dane o mieście do zmiennych)*
18. Wyświetl nazwy miast, usuwając spacje z początku i końca. *(Podpowiedź: użyj funkcji TRIM )*
19. Wyodrębnij dzień tygodnia (jako liczbę lub nazwę), w którym zostało założone każde miasto. *(Podpowiedź: użyj EXTRACT(DOW FROM ...) lub to\_char )*
20. Napisz zapytanie, które wyświetli nazwę miasta i informację, czy liczba mieszkańców jest parzysta czy nieparzysta. *(Podpowiedź: użyj operatora % 2 i instrukcji CASE )*
21. Stwórz funkcję kraj\_z\_waluta(nazwa\_kraju\_param text) , która zwraca tekst: "Kraj [nazwa] ma walutę [symbol]". *(Podpowiedź: funkcja powinna przyjmować tekst i zwracać sformatowany tekst po pobraniu danych z tabeli kraje )*
22. Stwórz funkcję liczba\_miast\_w\_kraju(id\_kraju\_param int) , która zwraca liczbę miast przypisanych do danego kraju. *(Podpowiedź: użyj SELECT COUNT(\*) INTO ... FROM miasta WHERE id\_kraju = ... )*